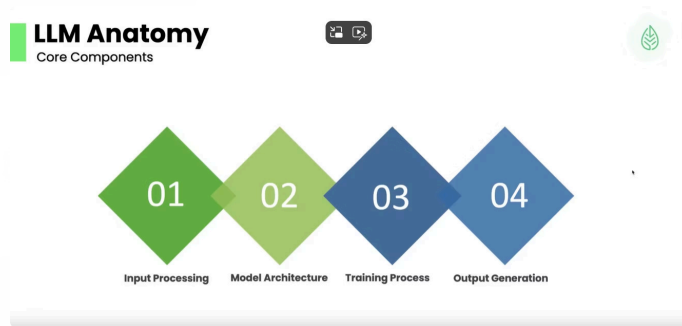




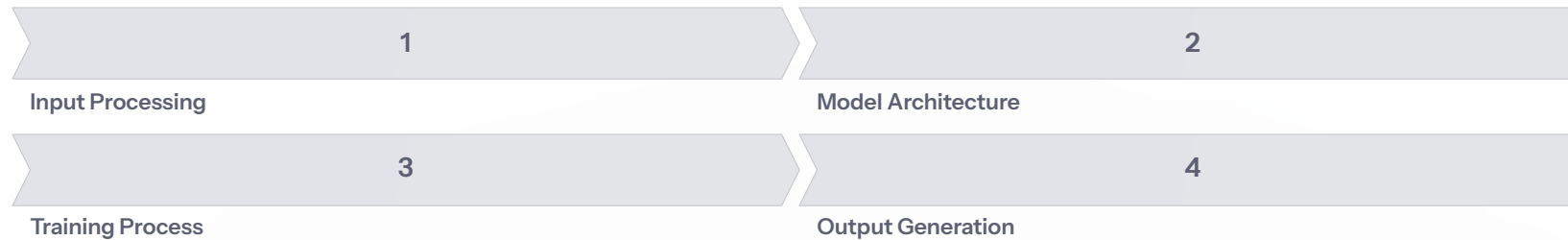
Testleaf

Always Ahead



LLM Anatomy

A deep dive into how Large Language Models actually work — from the atomic token all the way through transformers, embeddings, attention, and temperature. Whether you're building with LLMs or just curious about the magic under the hood, this guide breaks it down clearly.



How Does a Large Language Model Work?

At its core, an LLM is a **next-token prediction machine**. Given a sequence of tokens as input, it assigns probabilities to every possible next token and picks from the top candidates. It's less "thinking" and more a very sophisticated autocomplete — but at enormous scale.

The Input Side

Each word (or part of a word) is converted into a **vector representation** — a list of numbers that captures its meaning in high-dimensional space. These vectors are what the model actually processes.

The Output Side

The model outputs a probability distribution over the entire vocabulary. "mat" might be 85% likely, "tree" 10%, and everything else splits the remaining 5%. The model samples from this distribution to generate text one token at a time.

 A **token** is the atomic unit LLMs work in — roughly a part of a word, a whole short word, or a punctuation mark.

Tokens: The Atomic Unit of LLMs

Before an LLM can process any text, that text must be broken into **tokens**. Tokens don't map neatly to words — they follow their own rules based on how the model was trained. Understanding tokens helps you write better prompts and predict costs.

Words Can Have Multiple Tokens

Long or uncommon words like "Selenium" may be split across 2–3 tokens.

Tokens Can Be Part of a Word

Subword tokenization means prefixes, suffixes, and roots are often their own tokens.

Spaces & Punctuation Count

Every space, comma, and period is a token too — they all add up.

Not Fixed to Characters

Token boundaries are learned statistically, not based on fixed character counts.

Text	Approx. Tokens
"Hello world!"	3 tokens
"The quick brown fox jumps over the lazy dog."	9-10 tokens
"Selenium WebDriver automates browsers."	5-6 tokens

Tokens & Cost: Why It Matters

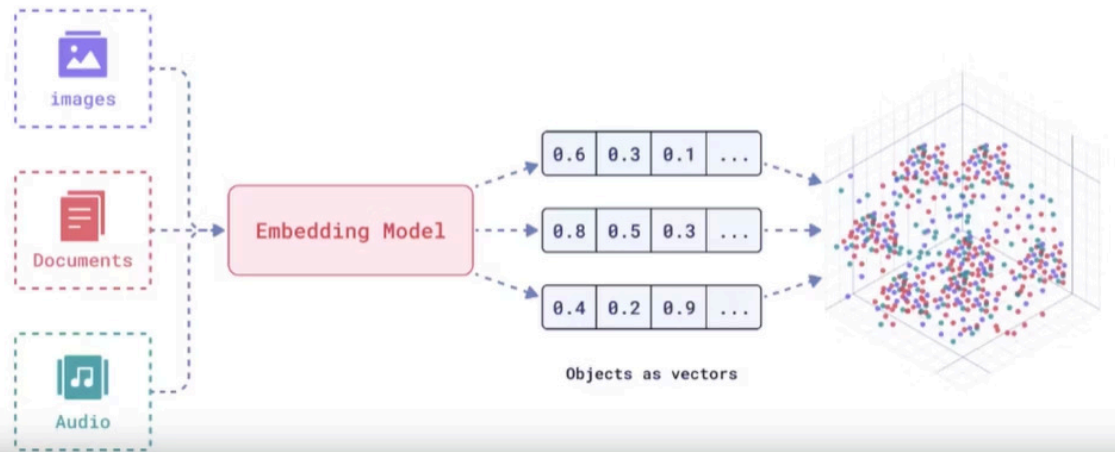
Token count isn't just a technical detail — it's directly tied to **how much you pay** when using an LLM API. Most providers charge per input + output token. Being deliberate about your `max_tokens` setting keeps costs predictable and responses appropriately scoped.

Use Case	Recommended <code>max_tokens</code>
✓ Single sentence answers	50–100
✓ Short summaries	150–250
✓ Detailed explanations	300–500
✓ Generating code snippets	500–1000
✓ Long-form articles	1000–2048



Setting `max_tokens` too low can cut off responses mid-sentence. Too high, and you'll burn budget on padding. Match it to your use case.

Embeddings



Embeddings: Meaning as Math

An **embedding** is a way of representing any piece of content — text, images, audio — as a list of numbers (a vector). These vectors live in a high-dimensional space where **similar concepts cluster together**. "King" and "Queen" end up near each other; "cat" and "automobile" don't.

Why Vectors?

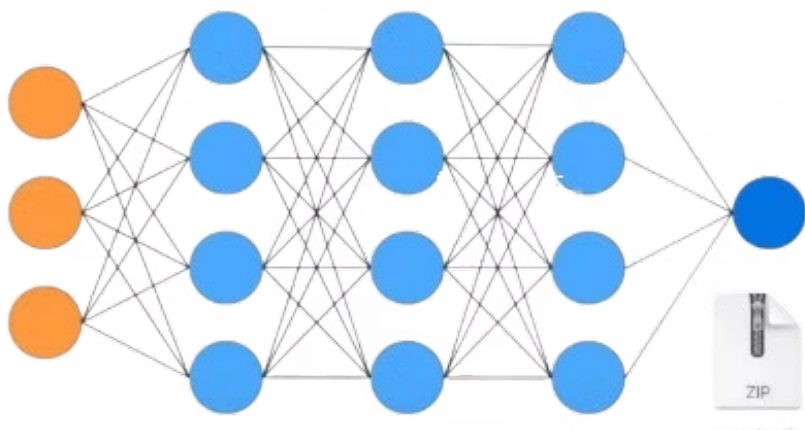
Math works on numbers, not words. Converting meaning to vectors lets the model perform operations — like comparing relevance or combining context — using standard linear algebra.

Multi-modal Power

The same embedding approach works across modalities. Images, documents, and audio can all be projected into a shared vector space, enabling cross-modal search and reasoning.

LLaMA: Large Language Model Meta AI

LLaMA is Meta's family of open-weight language models. Like all LLMs, it takes a sequence of input tokens, converts them to vectors, passes them through a neural network, and outputs a probability distribution over the next token. What makes LLaMA notable is its **efficiency** — strong performance at smaller parameter counts.



The neural network layers learn to compress the statistical patterns of language into **parameters** — billions of numbers stored in a weights file.

The Prediction Loop

- Input tokens are embedded into vectors
- Vectors flow through transformer layers
- The final layer outputs token probabilities
- "mat" → 85%, "tree" → 10%, others → 5%
- Sample → append token → repeat

📄 The model's "knowledge" lives entirely in its weights — the billions of parameters trained on vast text corpora.

Transformers

Before Transformer works !!

Input

"I went to the bank to deposit money."

Token

["I", "went", "to", "the", "bank", "to", "deposit", "money"]

Embeddings

"bank" → [0.7, 0.2, 0.4]
"deposit" → [0.1, 0.8, 0.3]
"money" → [0.3, 0.6, 0.5]

Transformers: Before the Magic Happens

Before the transformer architecture came along, models processed text **sequentially** — word by word. This made it hard to relate words far apart in a sentence. Transformers changed everything by processing all tokens simultaneously and learning relationships between any two positions at once.

The first step is still familiar: tokenize the sentence, then embed each token into a vector. The sentence *"I went to the bank to deposit money."* becomes tokens, and key words like "bank," "deposit," and "money" each get their own embedding vectors that capture their rough meaning — before context is applied.

 The crucial insight: embeddings at this stage don't yet know that "bank" means a **financial institution** rather than a riverbank. That disambiguation happens in the next steps.

Transformers

Self Attention Blocks



Input

"I went to the bank to deposit money."

Token

["I", "went", "to", "the", "bank", "to", "deposit", "money"]

Embeddings

"bank" → [0.7, 0.2, 0.4]

"deposit" → [0.1, 0.8, 0.3]

"money" → [0.3, 0.6, 0.5]

The model asks:

"What does '**bank**' mean in this sentence? Should I focus on 'deposit' and 'money,' or on 'river' and 'water'?"

Transformers: Self-Attention Blocks

The key innovation in transformers is **self-attention**: the ability for every token to "look at" every other token in the sequence and ask, *"How relevant are you to understanding me?"*

In our example sentence, the model asks: *"What does 'bank' mean in this sentence? Should I focus on 'deposit' and 'money,' or on 'river' and 'water'?"* Self-attention lets the model answer this question by comparing embeddings across the full context window — simultaneously, in parallel.

Query

What am I looking for?

Key

What do I have to offer?

Value

What information do I pass along?

Transformers

Step 1: Compare "bank" to Every Other Word

Input

"I went to the bank to deposit money."

Token

["I", "went", "to", "the", "bank", "to", "deposit", "money"]

Embeddings

"bank" → [0.7, 0.2, 0.4]

"deposit" → [0.1, 0.8, 0.3]

"money" → [0.3, 0.6, 0.5]

The model compares "bank" to all other words in the sentence to see how relevant they are.

For example:

"bank" vs. "deposit" → High relevance (financial context).

"bank" vs. "money" → High relevance (financial context).

"bank" vs. "river" → Low relevance (not in this sentence).

Transformers: Step 1 — Compare to Every Other Word

The model compares "bank" to all other words in the sentence to measure how relevant each one is. This comparison happens using dot products between the query vector of "bank" and the key vectors of every other token.

"bank" vs. "deposit"

● High relevance — financial context

"bank" vs. "money"

● High relevance — financial context

"bank" vs. "river"

● Low relevance — not in this sentence

This comparison produces a raw score for each word pair. The higher the score, the more that word should influence how "bank" is interpreted. These scores get normalized in the next step.

Transformers

Step 2: Assign Attention Scores

Input	"I went to the bank to deposit money."
Token	["I", "went", "to", "the", "bank", "to", "deposit", "money"]
Embeddings	"bank" → [0.7, 0.2, 0.4] "deposit" → [0.1, 0.8, 0.3] "money" → [0.3, 0.6, 0.5]

The model assigns **attention scores** to each word based on how relevant they are to "bank."

"deposit" → High score (e.g., 0.8).
"money" → High score (e.g., 0.7).
"went" → Low score (e.g., 0.1).
"the" → Very low score (e.g., 0.05).

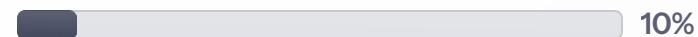
Transformers: Step 2 — Assign Attention Scores

Raw comparison scores are converted into **attention scores** using a softmax function — which normalizes them into a probability distribution that sums to 1. Higher scores mean the model pays more "attention" to that word when deciding what "bank" means.



"deposit"

High attention — strongly financial



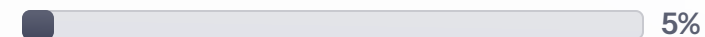
"went"

Low attention — mostly grammar



"money"

High attention — reinforces finance



"the"

Very low — function word

Transformers

Step 3: Combine Information



Input

"I went to the bank to deposit money."

Token

["I", "went", "to", "the", "bank", "to", "deposit", "money"]

Embeddings

"bank" → [0.7, 0.2, 0.4]
"deposit" → [0.1, 0.8, 0.3]
"money" → [0.3, 0.6, 0.5]

The model combines the information from all the words, weighted by their attention scores.

Since "deposit" and "money" have high scores, the model understands that "bank" refers to a **financial institution**.

Transformers: Step 3 — Combine Information

The final step of self-attention: **weighted aggregation**. The model takes each word's value vector and multiplies it by its attention score, then sums everything together. This produces a new, context-aware representation of "bank."

Since "deposit" and "money" have high scores, their information flows heavily into the updated "bank" vector. The model now understands that **"bank" refers to a financial institution** — not a riverbank — in this particular sentence.

- ✓ This context-sensitivity is what makes transformers so powerful. The same word can mean different things in different sentences, and the model handles this gracefully through attention.

Transformers

Feed-Forward Neural Network



Input

Example Sentence 1:

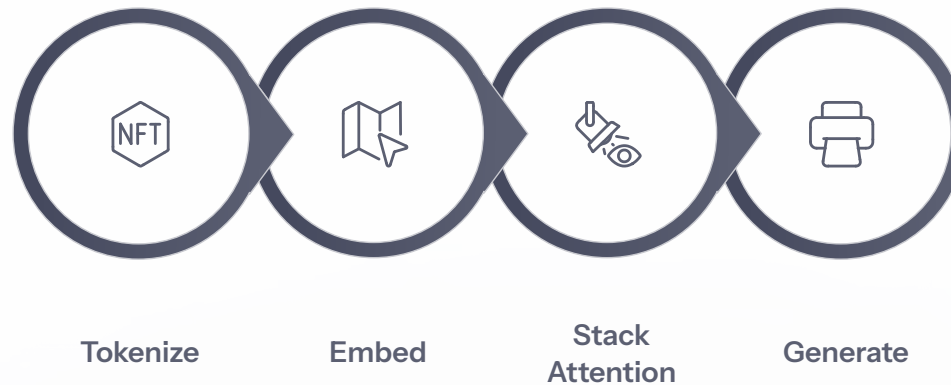
"I went to the bank to deposit money."

Example Sentence 2:

"I sat by the bank of the river."

The Full Transformer Pipeline

Self-attention is just one building block. A complete transformer layer wraps attention with normalization, feed-forward networks, and residual connections. These are stacked dozens of times — GPT-4, for instance, uses 96 layers — each pass refining the model's understanding of the input.



Each layer adds a richer level of abstraction — from syntax at early layers to complex semantics and reasoning at deeper layers. By the time the final layer runs, the model has a highly contextual representation ready to generate the next token.

Transformers

Feed-Forward Neural Network



Input

Example Sentence 1:

"I went to the bank to deposit money."

Example Sentence 2:

"I sat by the bank of the river."

The model further refines the representation of each word.

Output of Layer 2 (after attention block):

For Sentence 1: "bank" is now even more strongly associated with **finance**.

For Sentence 2: "bank" is now even more strongly associated with a **river**.

Training: How the Model Learns

LLMs learn by reading enormous amounts of text and repeatedly making predictions. During training, the model sees a sequence of tokens, predicts the next one, then checks whether it was right. The difference between its prediction and the actual next token — called the **loss** — is used to slightly adjust all the model's parameters.

This process, called **backpropagation**, happens billions of times across trillions of tokens. Over time, the model's parameters encode the statistical patterns of human language at a staggering level of detail.

- ❏ Training a frontier model like GPT-4 or LLaMA 3 costs tens of millions of dollars in compute. Fine-tuning a pre-trained model for a specific task is far cheaper — and often just as effective.

Transformers

Feed-Forward Neural Network

Input

Example Sentence 1:

"I went to the bank to deposit money."

Example Sentence 2:

"I sat by the bank of the river."

Sentence	After Self-Attention	After FFN
"I went to the bank to deposit money."	[0.9, 0.1, 0.3] (financial)	[0.95, 0.05, 0.1] (even stronger finance)
"I sat by the bank of the river."	[0.2, 0.8, 0.4] (river)	[0.1, 0.9, 0.3] (even stronger river)

RLHF: Teaching the Model to Be Helpful

Raw pre-training produces a model that's great at completing text — but not necessarily at following instructions or being helpful. **Reinforcement Learning from Human Feedback (RLHF)** is the technique used to align models with human preferences.

- 01

Generate Responses

The model produces multiple responses to the same prompt.
- 02

Human Ranking

Human raters rank the responses from best to worst.
- 03

Reward Model

A separate model learns to predict which responses humans prefer.
- 04

Fine-tune with RL

The LLM is updated to produce responses that score higher on the reward model.

Transformers



Input → Tokenization → Embeddings → [Self-Attention → FFN] × N layers → Output

Context Window: The Model's Working Memory

The **context window** is the maximum number of tokens the model can "see" at once — its working memory. Everything outside this window is invisible to the model. Earlier models had context windows of just 2K–4K tokens. Modern models like Gemini 1.5 Pro support up to **1 million tokens**.

What Goes In the Context?

- System prompt / instructions
- Full conversation history
- Retrieved documents (RAG)
- Tool call results
- The model's own prior outputs

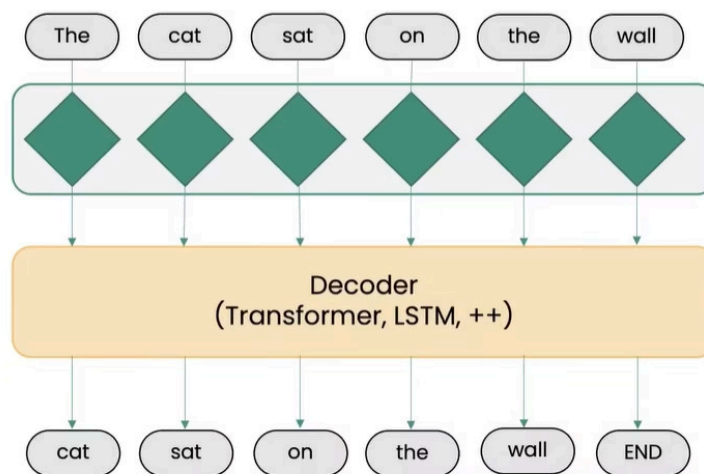
Why It Matters

A longer context window lets the model reason over more information at once — entire codebases, long documents, or extended conversations — without losing track of earlier content. But longer contexts also cost more tokens.

Pre-Training and Fine-Tuning

Step 1: Pre-Train (On Language Modeling)

Lots of text; learn general things!



Prompt Engineering: Shaping the Output

Prompting is the primary interface between humans and LLMs. The words you choose — and how you structure them — have an outsized effect on quality. A well-crafted prompt can turn a mediocre response into an exceptional one without changing a single model parameter.



Role / Persona

Tell the model who it should be. "You are a senior software engineer reviewing a pull request."



Clear Task

State exactly what you want. Vague instructions produce vague output. Be specific about format and scope.



Few-Shot Examples

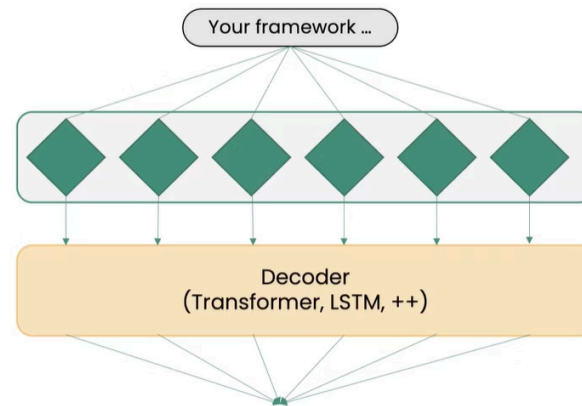
Show the model what "good" looks like with 2–3 examples before asking for the real output.

Pre-Training and Fine-Tuning



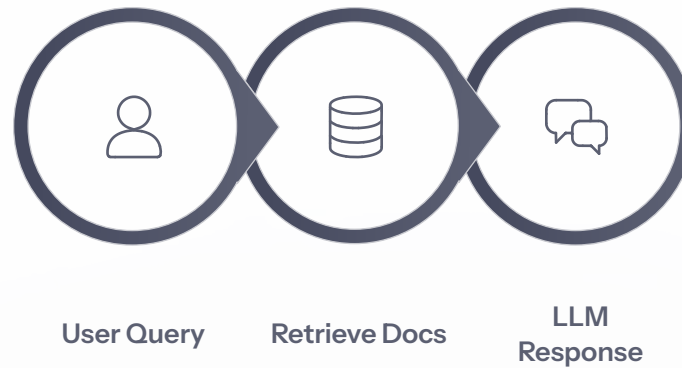
Step 2: Fine-Tune (On Your Task)

Not many labels;
adapt to your task !!



RAG: Retrieval-Augmented Generation

RAG solves one of LLMs' biggest limitations: they don't know anything that happened after their training cutoff, and they can't access your private data. RAG fixes this by retrieving relevant documents at inference time and stuffing them into the context window before generating a response.



The result is a model that can answer questions about your internal knowledge base, recent news, or any other corpus — without expensive retraining. RAG is the backbone of most enterprise LLM applications today.

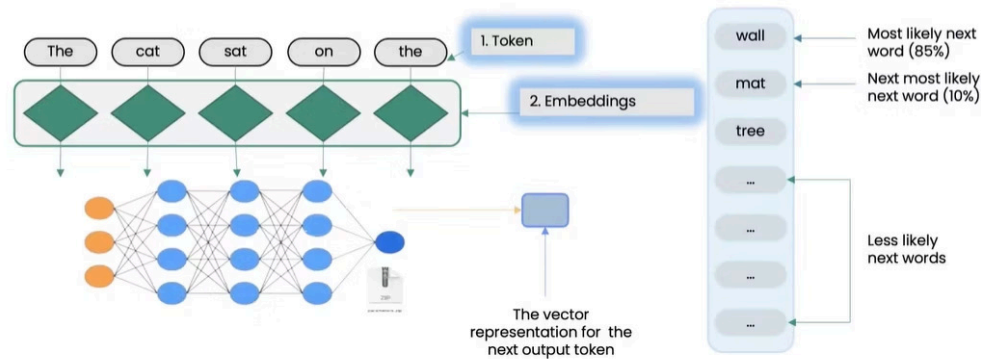
Output Generation

Decoding Strategies

Greedy Search: Picks the most likely word at each step.

Beam Search: Considers multiple possibilities.

Sampling: Randomly selects words based on probabilities.



Hallucinations: When Models Make Things Up

LLMs can generate text that sounds confident and authoritative — but is completely wrong. This is called a **hallucination**. It happens because the model is optimized to produce plausible-sounding text, not necessarily true text. It doesn't "know" what it doesn't know.

Why It Happens

- The model interpolates patterns, not facts
- No internal truth-checking mechanism
- Rare or ambiguous facts get "filled in"
- Training data may contain errors

How to Mitigate It

- Use RAG to ground responses in real documents
- Ask the model to cite sources
- Use lower temperature for factual tasks
- Add verification layers or human review

⊗ Never deploy an LLM in a high-stakes context (medical, legal, financial) without a robust verification and human-in-the-loop strategy.

Use Case	Best Temperature	Reason
✔ Structured Responses (JSON, Code, Facts)	0.0 – 0.2	Ensures consistent & correct output
✔ Technical Documentation, Tutorials	0.3 – 0.5	Slight flexibility, but still structured
✔ Chatbots, Support Agents	0.6 – 0.8	More natural responses with some variation
✔ Creative Writing, Brainstorming	0.9 – 1.2	Encourages diverse, creative ideas
✔ Marketing, Advertisements	1.2 – 1.5	Generates unique, engaging content

Temperature: Dialing Creativity Up or Down

Temperature controls how random the model's sampling is. It scales the probability distribution before the model picks the next token. A low temperature makes the model more deterministic — it'll almost always pick the highest-probability token. A high temperature flattens the distribution, giving lower-probability (more surprising) tokens a real chance.

Temperature = 0 Fully deterministic. Always picks the most likely token. Great for code, math, and factual Q&A.	Temperature = 0.7 Balanced. Natural-sounding, slightly varied. The sweet spot for most general use cases.	Temperature = 1.5+ Very creative — bordering on chaotic. Useful for brainstorming, poetry, or exploring unexpected ideas.
---	---	---

Feature	Description
✔ Content Filtering	Detects hate speech, violence, abuse, misinformation, etc.
✔ Customizable	Can be fine-tuned to align with company policies .
✔ Works with LLaMA	Designed for Meta's LLaMA models , ensuring native compatibility .
✔ Real-Time Moderation	Filters AI responses before they reach users .
✔ Compliance & Safety	Ensures AI-generated content follows legal and ethical standards .

Moderation with LlamaGuard

LlamaGuard is Meta's open-source safety classifier designed to moderate both inputs and outputs in LLM pipelines. It's itself a fine-tuned LLM — trained specifically to detect unsafe content across a range of categories, from hate speech to self-harm, violence, and privacy violations.

How It Works

LlamaGuard takes a prompt-response pair and classifies whether either the user input or the model's output violates safety policies. It returns a simple safe / unsafe label plus the violated category.

Where to Use It

- Screen user inputs before they reach your LLM
- Filter model outputs before they reach users
- Audit conversation logs for policy violations
- Customize safety policies for your specific domain

✔ LlamaGuard is open-weight and can be self-hosted, giving teams full control over their moderation pipeline without sending data to a third-party API.